Build Your Own Clerk in Go: Stages, Steps and Learning Plan

Oct 17, 2026 · @kEYS

You will build a hosted identity service in Go, the same kind of product as Clerk, in 13 stages (0 to 12). Developers plug it into their apps instead of writing signup, login, sessions and 2FA themselves.

Each stage below has the same four parts: **Goal**, **What to learn**, **Steps**, and **You pass when**. Do not move on until you can tick every box under "You pass when".

Overview

What the finished product does

- End users sign up, sign in, reset passwords, use Google or GitHub login, and turn on authenticator-app 2FA.
- Developers create an application in your dashboard, get API keys, and verify user sessions in their own backend with your Go SDK.
- Developers manage users, organizations and roles through your API and dashboard.

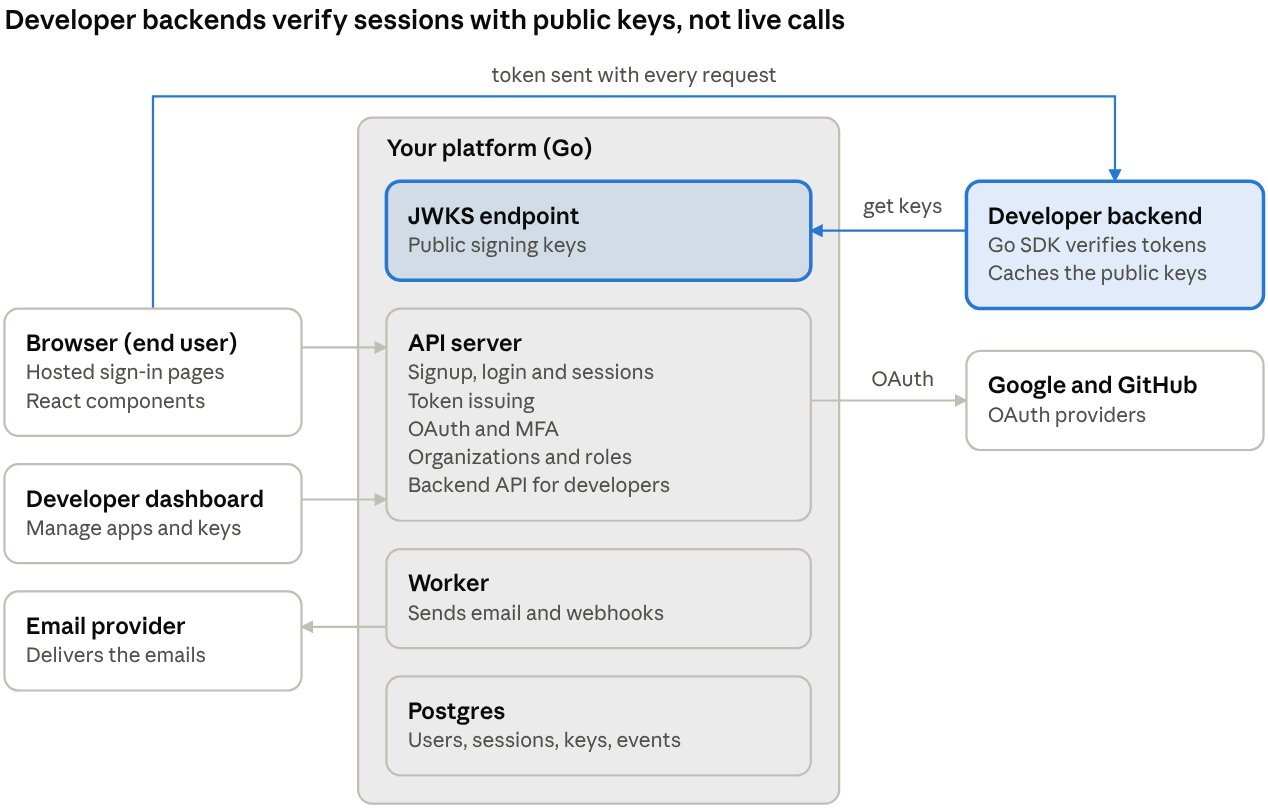
What you are deliberately not building first

- Enterprise SAML single sign-on, passkeys and bot detection. They are stretch goals after Stage 12.
- A custom email delivery system. Use an email provider's API (Resend, Postmark or SES) or a local mail catcher in development.
- Billing. Add it only if you want to turn this into a real business.

Ground rule

This is a learning build. For a real product with real users, have an experienced security engineer review it before launch.

Architecture and tech stack



platform architecture · 3 sides, 1 verification path

The highlighted path is the heart of the product: the end user's browser sends a signed token to the developer's backend, which checks it with public keys fetched from your JWKS endpoint. Your API server is only called at sign-in and token refresh.

Recommended stack

Layer	Choice	Why
Language	Go 1.22 or newer	Strong standard library, easy concurrency, single-binary deploys
HTTP router	`net/http` with `chi`	Small, close to the standard library
Database	PostgreSQL with `pgx`	Transactions, constraints and row-level features you will use

Layer	Choice	Why
Migrations	`goose` or `golang-migrate`	Versioned, reversible schema changes
Passwords	`golang.org/x/crypto/argon2`	Current recommended password hash
Tokens	`lestrrat-go/jwx` or `golang-jwt/jwt`	JWT and JWKS support
OAuth	`golang.org/x/oauth2` and `coreos/go-oidc`	Standard flows and ID token validation
Email	Mailpit locally, Resend or Postmark in production	Easy to test, then easy to send
Queue and cache	Postgres first, Redis later	Fewer moving parts to start
Frontend	TypeScript, React, Next.js or Vite	Browsers need JavaScript for components
Deploy	Docker, then Fly.io, Railway or a VPS	Simple and cheap to learn on

Foundations to learn before Stage 0

Spend one to two weeks here. You do not need to master these, only reach the "ready when" check in each row.

Topic	What to learn	Ready when you can
Go language	Structs, interfaces, errors, slices and maps, packages and modules, `context`, goroutines and channels, the `testing` package	Write a small CLI tool with tests and explain when to use a pointer receiver
HTTP and the web	Methods, status codes, headers, cookies, JSON bodies, CORS, how browsers send cookies	Build a JSON API with `net/http` and call it with `curl`
SQL and Postgres	Tables, primary and foreign keys, indexes, joins, transactions, unique constraints, migrations	Design a two-table schema and write queries with joins from Go using `pgx`

Topic	What to learn	Ready when you can
Cryptography basics	Hashing vs encryption vs signing, salts, HMAC, symmetric vs asymmetric keys, random number generation	Explain why a password hash needs a salt and why `math/rand` is wrong for tokens
Web security basics	XSS, CSRF, SQL injection, the same-origin policy, cookie flags (`HttpOnly`, `Secure`, `SameSite`)	Describe how each attack works and which defence stops it
Docker	Images, containers, `docker compose`	Run Postgres locally with one command
Git	Branches, commits, pull requests	Keep every stage on its own branch with a clear history

Recommended order

1. Go: *A Tour of Go*, then *Go by Example*, then build one small HTTP API.
2. SQL: work through a Postgres tutorial, then read the `pgx` docs.
3. Security: read the OWASP Top 10 and the OWASP Cheat Sheet Series pages on authentication, session management and password storage.
4. Cryptography: read how HMAC and digital signatures work at the level of ideas, not the maths.

Stage 0: Project setup

Goal: a running Go server connected to Postgres, with migrations and a test setup, so every later stage starts from solid ground.

What to learn

- Go modules and a clean project layout (`cmd/`, `internal/`, `migrations/`).
- Configuration from environment variables (the twelve-factor approach).
- Database migrations with `goose` or `golang-migrate`.
- Structured logging with `log/slog`.
- Writing HTTP handler tests with `httptest`, and database tests against a real Postgres.

Steps

1. Create the repo and run `go mod init`. Suggested layout: `cmd/api/main.go`, `internal/auth`, `internal/store`, `internal/http`, `migrations/`.
2. Add `docker-compose.yml` with Postgres and a mail catcher such as Mailpit.
3. Load config (database URL, port, secrets) from environment variables with a small config struct.
4. Connect to Postgres with `pgx` and a connection pool.
5. Write your first migration, run it, and add a `/healthz` endpoint that pings the database.
6. Add structured request logging and a request ID middleware.
7. Add a `Makefile` or `Taskfile` with `run`, `test`, `migrate-up` and `migrate-down`.
8. Set up GitHub Actions to run `go vet` and `go test` on every push.

You pass when

- [ ] `docker compose up` then `make run` gives you a server that answers `/healthz`.
- [ ] Migrations go up and down cleanly on an empty database.
- [ ] One handler test and one database test pass in CI.

Stage 1: Users and password authentication

Goal: a user can sign up with email and password, and the server can check that password later without ever storing it.

What to learn

- Why passwords are hashed, not encrypted, and why the hash must be slow and salted.
- Argon2id: its memory, iterations and parallelism parameters (read the OWASP Password Storage Cheat Sheet for current recommended values).
- Constant-time comparison, and why `==` on secrets leaks information.
- Input validation: email format, password length rules, and why you normalise email to lower case.
- User enumeration: why "wrong password" and "no such user" must look identical to the caller.

Steps

1. Create a `users` table: `id` (UUID), `email` (unique), `password_hash`, `created_at`, `updated_at`.

2. Write a `password` package with `Hash(plain)` and `Verify(plain, hash)` using `golang.org/x/crypto/argon2`. Store the parameters inside the encoded hash string.
3. Write a user store with `Create` and `GetByEmail`, handling the unique-email conflict as a clean error.
4. Add `POST /v1/signup` and `POST /v1/login` (login only returns success or failure for now).
5. Return the same error message and a similar response time for unknown email and wrong password. Hash a dummy password when the user does not exist.
6. Add request validation and consistent JSON error responses.
7. Write table-driven tests for hashing and for both endpoints.

You pass when

- [ ] The database contains no plain-text passwords, only encoded hashes.
- [ ] Two users with the same password have different hashes.
- [ ] Signing up twice with the same email (any letter case) fails cleanly.
- [ ] Login errors for a missing user and a wrong password are identical.

Stage 2: Sessions and cookies

Goal: after login, the browser stays signed in, and the server can look up who is making each request, list a user's devices, and sign them out.

What to learn

- How cookies work: `Set-Cookie`, `Domain`, `Path`, `Expires`, and the `HttpOnly`, `Secure` and `SameSite` flags.
- Generating session tokens with `crypto/rand` (at least 32 random bytes), never `math/rand`.
- Storing only a hash (SHA-256) of the session token in the database, so a database leak does not leak live sessions.
- Idle timeout vs absolute timeout, and session rotation after login.
- Middleware in Go: wrapping handlers and passing values through `context`.

Steps

1. Create a `sessions` table: `id`, `user_id`, `token_hash`, `user_agent`, `ip`, `created_at`, `last_active_at`, `expires_at`, `revoked_at`.

2. On successful login, generate a random token, store its hash, and set it in a cookie with `HttpOnly`, `Secure` and `SameSite=Lax`.
3. Write `AuthMiddleware` that reads the cookie, hashes it, finds the live session, and puts the user in the request context.
4. Add `GET /v1/me`, `POST /v1/logout`, `GET /v1/sessions` and `DELETE /v1/sessions/{id}`.
5. Add idle and absolute expiry, and update `last_active_at` without writing to the database on every request (for example, at most once a minute).
6. Add a cleanup job that deletes expired sessions.
7. Test the flow end to end with a cookie jar: sign up, log in, call `/v1/me`, log out, call `/v1/me` again.

You pass when

- [ ] `/v1/me` returns the user with a valid cookie and 401 without one.
- [ ] After logout, the old cookie no longer works, even if copied before.
- [ ] The database never holds a raw session token.
- [ ] Revoking one session leaves the user's other sessions alive.

Stage 3: JWTs, JWKS and the Go verification SDK

Goal: a developer's own backend can verify "this request is from user X" using only your public keys, without calling your server on every request. This is the core of the Clerk model.

What to learn

- JWT structure: header, claims, signature. Standard claims: `iss`, `sub`, `aud`, `exp`, `iat`, `nbf`, `jti`.
- Asymmetric signing: RS256 or ES256 (or EdDSA), and why you sign with a private key and verify with a public one.
- JWKS (JSON Web Key Set, RFC 7517): publishing public keys at `/.well-known/jwks.json`, with a `kid` on each key.
- Key rotation: adding a new key before retiring the old one.
- The common JWT mistakes: accepting `alg: none`, trusting the header's algorithm, skipping `exp`, `iss` and `aud` checks.

- Short-lived access tokens with a longer-lived session behind them (the pattern Clerk uses).

Steps

1. Create a `signing_keys` table (or load keys from files) with `kid`, algorithm, public key, encrypted private key, `created_at`, `retired_at`.
2. Issue a short-lived JWT (about 60 seconds) from `POST /v1/sessions/{id}/token`. Include `sub` (user ID), `sid` (session ID), `iss` and `exp`.
3. Serve public keys at `/.well-known/jwks.json`.
4. Create a separate Go module, `yourclerk-go`, with `VerifyToken(ctx, token)` that fetches and caches the JWKS, checks signature, `exp`, `iss` and `aud`, and returns the claims.
5. Add an `http.Handler` middleware to that SDK: `RequireAuth(next)`.
6. Build a tiny demo app that uses the SDK to protect a route.
7. Implement key rotation with two active keys, and test that old tokens still verify until they expire.
8. Write negative tests: tampered payload, wrong key, expired token, wrong issuer, and `alg: none`.

You pass when

- [ ] The demo app accepts a fresh token and rejects each of the five bad tokens above.
- [ ] Rotating the signing key causes no failed requests for users with live tokens.
- [ ] The SDK works with no shared secret and no call to your API on the hot path.
- [ ] Revoking a session stops new tokens from being issued within one token lifetime.

Stage 4: Multi-tenancy, applications and API keys

Goal: many developers can each create their own application inside your system, with fully separate users, settings and keys. This is what turns an auth library into a Clerk-style platform.

What to learn

- Multi-tenant data models: a shared database with an `application_id` on every row, vs a schema or database per tenant. Start with the shared model.
- Composite uniqueness: email must be unique per application, not globally.

- Two kinds of caller: **end users** (cookies and session tokens) and **developers' servers** (secret API keys).
- API key design: a readable prefix (`sk_live_`, `pk_live_`), a random secret, storage as a hash, and showing the full key only once.
- Request scoping: resolving the application from a publishable key or a domain, then carrying it in `context`.

Steps

1. Add `developers` (the people who use your dashboard) and `applications` tables, with each application owned by a developer.
2. Add `application_id` to `users`, `sessions` and `signing_keys`. Change the email unique constraint to `(application_id, email)`.
3. Give each application a publishable key (safe in the browser) and a secret key (server only). Store only a hash of the secret key.
4. Write middleware that resolves the application on every request and rejects unknown keys.
5. Make every store query take an `application_id`, so a missing filter is a compile error rather than a data leak.
6. Give each application its own signing keys and its own JWKS URL, and its own issuer value.
7. Add per-application settings, such as allowed origins, session lifetime and sign-up on or off.
8. Write a test with two applications containing the same email, and prove neither can see or affect the other.

You pass when

- [ ] Two applications can each have a user with the same email and different passwords.
- [ ] A token from application A fails verification when checked against application B.
- [ ] A leaked publishable key cannot call any secret-key endpoint.
- [ ] Every database query for tenant data is scoped by `application_id`.

Stage 5: Email verification and password reset

Goal: users prove they own their email address, and can recover a lost password safely.

What to learn

- Single-use, expiring tokens: generate with `crypto/rand`, store a hash, delete on use.
- Sending email: SMTP vs a provider's HTTP API, and the basics of SPF, DKIM and DMARC so your mail is not marked as spam.
- Why reset flows are a favourite attack target: enumeration, token leakage through the `Referer` header, and reusable links.
- Background jobs: sending email outside the request, with retries. A simple database-backed queue is enough to start.
- Email templates: plain-text and HTML versions.

Steps

1. Create a `verification_tokens` table with `user_id`, `purpose` (verify or reset), `token_hash`, `expires_at`, `used_at`.
2. Add an `email_verified_at` column to `users`.
3. Write a `mailer` interface with two implementations: one that prints to the log (or Mailpit) and one that calls a real provider.
4. Send a verification email on signup. Add `POST /v1/verify-email` that consumes the token.
5. Add `POST /v1/password/forgot`. Always respond the same way whether or not the email exists.
6. Add `POST /v1/password/reset`. Check the token, set the new password, and revoke all of that user's existing sessions.
7. Add a background worker that sends email from a queue, with retry and backoff.
8. Test: expired token, reused token, token from another application, and a reset that invalidates old sessions.

You pass when

- [ ] A verification or reset link works exactly once and expires on time.
- [ ] The forgot-password response is identical for known and unknown emails.
- [ ] After a reset, every older session for that user stops working.
- [ ] A failed email send is retried and does not break signup.

Stage 6: OAuth 2.0 and OpenID Connect social login

Goal: users can sign in with GitHub and Google, and the same person is never turned into two accounts by accident.

What to learn

- OAuth 2.0 authorization code flow (RFC 6749), and why the implicit flow is obsolete.
- PKCE (RFC 7636): the code verifier and challenge, and why it protects the flow.
- The `state` parameter and what CSRF attack it stops.
- OpenID Connect on top of OAuth: ID tokens, the `openid email profile` scopes, the discovery document at `/.well-known/openid-configuration`.
- Validating an ID token: signature against the provider's JWKS, plus `iss`, `aud`, `exp` and `nonce`.
- Account linking risks: only link by email when the provider says the email is verified.

Steps

1. Register an OAuth app with GitHub, then with Google, and note the redirect URIs.
2. Create an `oauth_accounts` table: `user_id`, `provider`, `provider_user_id`, `email`, tokens (encrypted), unique on `(provider, provider_user_id)`.
3. Add `GET /v1/oauth/{provider}/start`. Generate `state`, `nonce` and a PKCE verifier, save them in a short-lived signed cookie, and redirect to the provider.
4. Add `GET /v1/oauth/{provider}/callback`. Check `state`, exchange the code with `golang.org/x/oauth2`, and fetch the profile (GitHub) or validate the ID token (Google).
5. Find or create the user, then create a normal session, so everything from Stages 2 and 3 keeps working.
6. Define the account-linking rule: sign in if the provider ID is known, link if the verified email matches an existing user, otherwise create a new user.
7. Store provider credentials per application, so each developer brings their own Google or GitHub client.
8. Wrap providers behind a `Provider` interface so adding a third one takes under an hour.
9. Test with a fake OAuth server (`httptest`) for success, bad `state`, bad `nonce`, and an unverified email.

You pass when

- [ ] Signing in with GitHub and with Google both create a working session.
- [ ] A tampered or missing `state` is rejected.
- [ ] A Google login with the same verified email as an existing password user links to that user, not a duplicate.

- [ ] Adding a new provider needs one new file implementing the interface.

Stage 7: Two-factor authentication (TOTP) and backup codes

Goal: users can protect their account with an authenticator app such as Google Authenticator, and recover with backup codes. This is the authenticator idea you wanted to build, now as a feature of the platform.

What to learn

- TOTP (RFC 6238) and HOTP (RFC 4226): HMAC-SHA1 over a time counter, dynamic truncation, 6-digit codes, 30-second steps.
- Base32 encoding for secrets, and the `otpauth://` URI format used in QR codes.
- Clock drift: accepting the previous and next time step.
- Replay protection: a code must work once only, so remember the last accepted time step.
- Encrypting secrets at rest with AES-GCM and a key kept outside the database.
- Multi-step login: a half-authenticated state after the password, before the second factor.
- Backup codes: random, single-use, stored as hashes.

Steps

1. Write your own TOTP generator and validator from the RFC. Verify it against the RFC 6238 test vectors, then against your phone's authenticator app.
2. Create a `mfa_factors` table: `user_id`, `type`, `secret_encrypted`, `verified_at`, `last_used_step`.
3. Add `POST /v1/mfa/totp/enroll`. Return the secret and an `otpauth://` URI, and render the QR code with a Go QR library.
4. Add `POST /v1/mfa/totp/verify` to finish enrolment: the user must enter one valid code before the factor turns on.
5. Change login into two steps: password check issues a short-lived "pending MFA" token, then the code check issues the real session.
6. Generate 10 backup codes at enrolment, show them once, and store only hashes. Using one marks it as spent.
7. Add `DELETE /v1/mfa/totp`, which requires a fresh code or password.
8. Test: wrong code, reused code, code from the previous step, a code from after the window, and a spent backup code.

You pass when

- [ ] Your implementation matches the RFC test vectors and your phone app.
- [ ] The same code cannot be used twice.
- [ ] A user with MFA on cannot get a full session with the password alone.
- [ ] The database never holds a TOTP secret in plain text.

Stage 8: Security hardening

Goal: the service resists the attacks that real authentication systems face every day: password guessing, credential stuffing, request forgery and abuse of your email sender.

What to learn

- Rate limiting algorithms: fixed window, sliding window, token bucket. Limiting by IP, by account and by application.
- Credential stuffing and brute force, and how lockouts and backoff help (and how lockouts can be abused to lock out a victim).
- CSRF: when it applies to cookie-based auth, and defences such as `SameSite` cookies, CSRF tokens and `Origin` checks.
- CORS: allowing only each application's registered origins.
- Security headers: `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`.
- Checking passwords against known-breached lists (the Have I Been Pwned range API uses k-anonymity).
- Audit logging: who did what, when, from where.
- Threat modelling: listing assets, attackers and attack paths before fixing anything.

Steps

1. Write a short threat model for your service and list the top ten risks. Keep it in the repo.
2. Add a rate limiter, first in memory, then backed by Redis or Postgres so it works across several servers.
3. Add per-account throttling with increasing delay after failed logins, and notify the user by email about repeated failures.
4. Add CSRF protection and strict CORS based on each application's allowed origins.
5. Add security headers to every response.

6. Reject breached passwords at signup and reset.
7. Add an `audit_events` table and record sign-ins, failures, password changes, MFA changes and session revocations.
8. Run `govulncheck`, `gosec` and `staticcheck` in CI, and fix what they find.
9. Write an abuse test script that fires 1,000 login attempts and confirm the limiter stops it.

You pass when

- [ ] Repeated failed logins are slowed or blocked, and the real user is emailed.
- [ ] A cross-site form post cannot perform an authenticated action.
- [ ] Every sensitive action leaves an audit row.
- [ ] CI fails if a known vulnerable dependency is added.

Stage 9: Organizations, memberships, roles and permissions

Goal: the apps built on your platform can support teams. Users belong to organizations, hold roles inside them, and can invite others.

What to learn

- Many-to-many data modelling with a `memberships` join table.
- Authorization models: role-based access control (RBAC) first, then permission-based checks. Know where attribute-based (ABAC) and relationship-based models (as in Google Zanzibar) fit, without building them yet.
- The difference between authentication (who are you) and authorization (what may you do), and which part your platform owns.
- Invitation flows: signed or random invite tokens, expiry, and what happens when the invitee has no account yet.
- Putting the active organization and role into the session token, and the staleness problem that follows.

Steps

1. Create `organizations`, `memberships` (`user_id`, `org_id`, `role`) and `invitations` tables, all scoped by `application_id`.
2. Add default roles `admin` and `member`, then let each application define custom roles with a list of permission strings such as `org:billing:manage`.

3. Add endpoints to create an organization, list the user's organizations, switch the active organization, and manage members.
4. Add invitation endpoints: create, resend, revoke and accept. Send the invite by email using your Stage 5 mailer.
5. Add `org_id`, `org_role` and `org_permissions` claims to the JWT.
6. Extend the Go SDK with `HasPermission(claims, "org:billing:manage")` and a middleware `RequirePermission(...)`.
7. Enforce rules in the service: the last admin cannot be removed, and users cannot grant roles higher than their own.
8. Test: a member cannot call an admin endpoint, a removed member's token stops working within one token lifetime, and an invitation works once.

You pass when

- [ ] One user can belong to several organizations with different roles in each.
- [ ] The SDK blocks a request that lacks the needed permission.
- [ ] An organization always keeps at least one admin.
- [ ] Invitations expire and cannot be reused.

Stage 10: Developer platform: Backend API, webhooks, SDKs and docs

Goal: a developer can integrate your product in under an hour, and keep their own database in sync with your user data.

What to learn

- Designing a public REST API: resource naming, pagination (cursor-based), idempotency keys, consistent errors, versioning.
- OpenAPI specifications, and generating docs and client code from them.
- Webhooks: event payloads, HMAC signatures with a timestamp, retries with exponential backoff, and delivery logs. Study how Stripe documents its webhooks.
- The outbox pattern: writing the event in the same database transaction as the change, so events are never lost.
- SDK design: small surface, good defaults, clear errors, semantic versioning.
- Writing documentation: a quickstart, concept guides and an API reference.

Steps

1. Create a secret-key-protected **Backend API**: list, get, create, update and delete users; list and revoke sessions; manage organizations and members.
2. Write an OpenAPI spec for it and serve interactive docs from that spec.
3. Add cursor pagination and `Idempotency-Key` support on create endpoints.
4. Create `webhook_endpoints` and `events` tables, plus an outbox worker that delivers events such as `user.created`, `session.created`, `session.revoked` and `organization.membership.created`.
5. Sign each delivery with HMAC-SHA256 over timestamp and body, and add retries with backoff and a delivery log.
6. Add a "resend event" action for developers whose endpoint was down.
7. Publish the Go SDK properly, with a README, examples and tags.
8. Write a second SDK for Node.js or Python to prove the API design works beyond Go.
9. Write the docs: a 10-minute quickstart that ends with a protected route in a sample app.
10. Ask one friend to follow the quickstart cold, and fix everything that confuses them.

You pass when

- [ ] A new developer completes the quickstart without asking you a question.
- [ ] A webhook receiver can verify signatures, and a stopped receiver gets the event after it restarts.
- [ ] Retrying a create request with the same idempotency key does not create a duplicate.
- [ ] The API reference is generated from the OpenAPI spec, not written by hand.

Stage 11: Dashboard and frontend components

Goal: developers manage their applications and users in a web dashboard, and their end users see ready-made sign-in screens. This is the only stage where most of the code is not Go.

What to learn

- TypeScript and React fundamentals: components, state, hooks, forms. Next.js or Vite are both fine.
- Calling your API from the browser with cookies, and what changes with CORS and `SameSite` across domains.

- Hosted auth pages vs embedded components, and the tradeoffs of each. Clerk offers both.
- Using a custom domain for the auth pages so cookies are first-party.
- Component library design: theming, accessible forms, error states, loading states.
- Frontend session handling: refreshing the short-lived token in the background before it expires.

Steps

1. Build the **dashboard** first: developer sign-in (use your own platform for it, which is a good test), create an application, view and copy API keys, and configure allowed origins and providers.
2. Add a **users table** with search, a user detail page, and actions to ban, delete and revoke sessions.
3. Add pages for organizations and members, webhook endpoints with a delivery log, and the audit log.
4. Build a hosted sign-in page at `https://<app>.accounts.yourdomain.com` that supports email and password, social buttons, the MFA step and password reset.
5. Publish a small React package with a `<ClerkProvider>`-style provider, `<SignIn />`, `<SignUp />`, `<UserButton />`, and hooks such as `useUser()` and `useSession()`.
6. Make the provider refresh the token in the background and expose a `getToken()` function.
7. Add a sample Next.js app that uses the package and calls a Go backend protected by your SDK. This is your end-to-end demo.
8. Check accessibility: keyboard navigation, labels and focus order on every form.

You pass when

- [ ] A brand-new developer creates an application and shows a working sign-in page in a sample app within 15 minutes.
- [ ] The sample app's Go backend rejects requests without a valid token.
- [ ] The dashboard can do every action the Backend API can do for users and sessions.
- [ ] All forms can be completed using only the keyboard.

Stage 12: Deployment, observability and reliability

Goal: the service runs on the public internet, survives failures, and tells you when something is wrong before your users do.

What to learn

- Containerising a Go service: multi-stage Dockerfiles and small final images.
- Hosting options: a single VPS with Docker Compose to start, then a managed platform (Fly.io, Railway, Render) or Kubernetes if you want that experience.
- TLS certificates (Let's Encrypt), reverse proxies (Caddy or nginx), and custom domains.
- Secrets management: keeping signing keys and encryption keys out of the repo and out of the image.
- Observability: structured logs, metrics (Prometheus), traces (OpenTelemetry), and alerting.
- Database operations: backups, point-in-time recovery, connection pooling, zero-downtime migrations.
- Load testing with `k6` or `vegeta`, and reading the results.

Steps

1. Write a multi-stage Dockerfile and build the image in CI.
2. Deploy to a server with a real domain and automatic HTTPS. Put the API, the dashboard and the hosted auth pages on separate subdomains.
3. Move all secrets into the host's secret store or environment, and document how to rotate each one.
4. Add Prometheus metrics for request rate, error rate, latency, login success and failure counts, and queue depth.
5. Add OpenTelemetry tracing across the HTTP handler, database and mailer.
6. Set up automated database backups, then practise restoring one to a fresh database.
7. Add graceful shutdown so in-flight requests finish during a deploy.
8. Load test login and token verification. Find the first bottleneck and fix it.
9. Write a runbook: how to rotate signing keys, revoke all sessions after a breach, and restore from backup.
10. Add a public status page or a simple uptime monitor with alerts.

You pass when

- [ ] You can deploy a new version with no failed requests.
- [ ] You have restored a backup into a clean database and the service worked against it.
- [ ] An alert fires within five minutes when login errors spike.

- [ ] You can state how many logins per second one instance handles, with a measurement to back it up.